

Университет ИТМО

Факультет программной инженерии и компьютерной техники

Лабораторная работа №1

по «Алгоритмам и структурам данных»

Базовые задачи

Выполнил:

Студент группы Р3206

Косогова М. А.

Преподаватели:

Косяков М.С.

Тараканов Д. С.

Санкт-Петербург

2026

Задача №J «Гоблины и очереди»

```
#include <iostream>
#include <list>
#include <vector>
using namespace std;

int main() {
    int N;
    cin >> N;
    if (N >= 1 && N <= 100000) {
        list<int> goblins;
        vector<int> result;
        auto mid = goblins.begin();
        int size = 0;
        for (int i = 0; i < N; ++i) {
            char oper;
            cin >> oper;
            if (oper == '+') {
                int id;
                cin >> id;
                goblins.push_back(id);
                size++;
                if (size == 1) {
                    mid = goblins.begin();
                } else if (size % 2 != 0) {
                    mid++;
                }
            } else if (oper == '*') {
                int id;
                cin >> id;
                if (goblins.empty()) {
                    goblins.push_back(id);
```

```

        size++;
        mid = goblins.begin();
    } else {
        auto next = mid;
        next++;
        goblins.insert(next, id);
        size++;
        if (size % 2 != 0) {
            mid++;
        }
    }
} else if (oper == '-') {
    if (!goblins.empty()) {
        result.push_back(goblins.front());
        goblins.pop_front();
        size--;
        if (size > 1) {
            if (size % 2 != 0) {
                mid++;
            }
        } else {
            mid = goblins.begin();
        }
    }
}
}
for (int id : result) {
    cout << id << endl;
}
} else {
    cout << "N read error!" << endl;
}
}

```

```
return 0;  
}
```

Пояснение к примененному алгоритму:

- 1) Считываем количество запросов N, создаем пустой список – очередь гоблинов, массив для ответов – ушедшие гоблины и указатель на текущую середину, а также переменную size для быстрого отслеживания размера и четности;
- 2) При команде «+» добавляем гоблина в конец списка, а если после этого размер очереди становится нечетным, сдвигаем указатель середины на один шаг вправо;
- 3) При команде «*» вставляем нового гоблина сразу после указателя середины и при нечетном размере также сдвигаем его на шаг вправо;
- 4) При команде «-» удаляем первого гоблина из начала очереди, запоминаем его номер в ответ и смещаем указатель середины, чтобы он оставался в центре после сдвига элементов (если размер стал больше 1 и нечетным – сдвигаем указатель вправо, иначе сбрасываем его на начало;
- 5) После обработки всех команд выводим сохраненные номера гоблинов в том порядке, в котором они уходили к шаманам.

Задача №L «Минимум на отрезке»

```
#include <deque>
```

```
#include <iostream>
```

```
int main() {  
    int N;  
    int K;  
    std::cin >> N >> K;  
    if (N >= 1 && N <= 150000 && K >= 1 && K <= 10000) {  
        if (K > N) {  
            K = N;  
        }  
        int* arr = new int[N];  
        for (int i = 0; i < N; i++) {  
            std::cin >> arr[i];  
        }  
    }  
}
```

```

std::deque<int> indexqu;
for (int i = 0; i < K; i++) {
    while (!indexqu.empty() && arr[indexqu.back()] >= arr[i]) {
        indexqu.pop_back();
    }
    indexqu.push_back(i);
}
std::cout << arr[indexqu.front()];
for (int i = K; i < N; i++) {
    if (!indexqu.empty() && indexqu.front() <= i - K) {
        indexqu.pop_front();
    }
    while (!indexqu.empty() && arr[indexqu.back()] >= arr[i]) {
        indexqu.pop_back();
    }
    indexqu.push_back(i);
    std::cout << " " << arr[indexqu.front()];
}
std::cout << std::endl;
delete[] arr;
} else {
    std::cout << "N K read error!" << std::endl;
}
return 0;
}

```

Пояснение к примененному алгоритму:

- 1) Считываем количество чисел и размер «окна», создаем массив и заполняем его значениями;
- 2) Используем двустороннюю очередь, которая хранит индексы элементов так, что значения по этим индексам идут по возрастанию, первый элемент очереди всегда указывает на индекс минимального элемента в текущем окне;
- 3) При добавлении нового элемента удаляем с конца очереди все индексы, значения которых больше или равны текущему, так как они уже не смогут стать минимумом;

- 4) При сдвиге окна проверяем первый элемент очереди, то есть, если его индекс вышел за левую границу окна, удаляем его;
- 5) После каждого шага минимумом является элемент массива с индексом в начале очереди, его и выводим.